Internacionalna
poslovno-informaciona akademija Tuzla
Visoka škola za savremeno poslovanje, informacione tehnologije i tržišne komunikacije

Uvod u analizu podataka

Predavanje 2

mr. Selena Kurtić, dipl.ing.el.

2. PREDAVANJE

Osnove Pythona za analizu podataka I
(Jupyter Notebook, varijable, petlje, funkcije)

Pronalaženje ključnih konektora

- Prvi Vam je dan na poslu u **DataSciencesteru**, a potpredsjednik za umrežavanje pun je pitanja o Vašim korisnicima. Do sada nije imao koga pitati, pa je jako uzbuđen što ste s njim.
- Posebno želi da identificirate ko su „ključni konektori“ među znanstvenicima podataka. U tu svrhu daje vam pregled cijele DataSciencester mreže.
- (U stvarnom životu, ljudi vam obično ne daju podatke koji su vam potrebni.)

Pronalaženje ključnih konektora

- Kako izgleda ovaj pregled podataka?

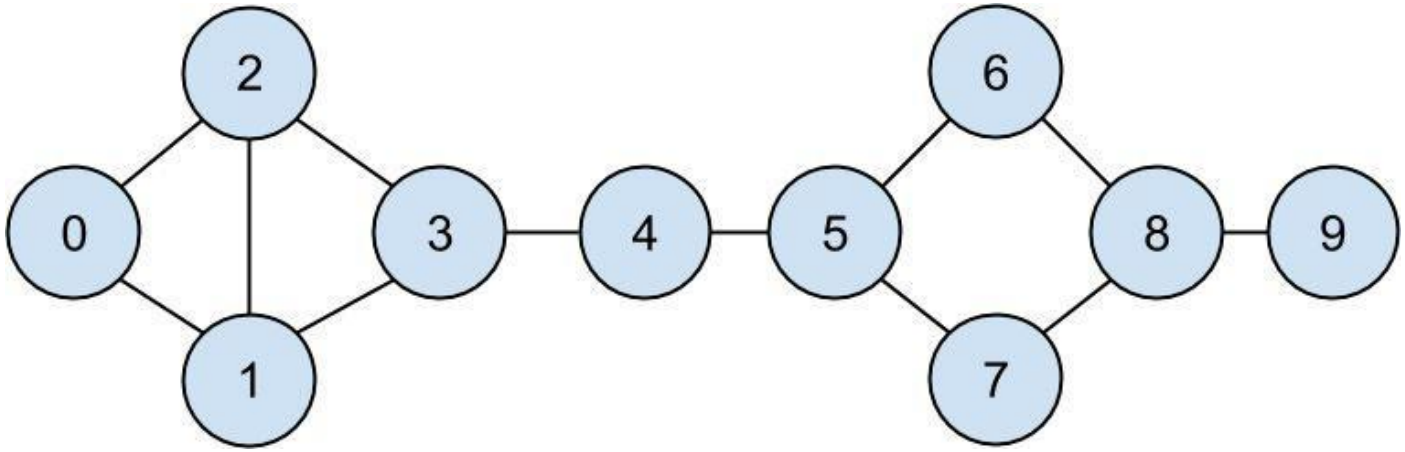
```
users = [
 { "id": 0, "name": "Hero" },
 { "id": 1, "name": "Dunn" },
 { "id": 2, "name": "Sue" },
 { "id": 3, "name": "Chi" },
 { "id": 4, "name": "Thor" },
 { "id": 5, "name": "Clive" },
 { "id": 6, "name": "Hicks" },
 { "id": 7, "name": "Devin" },
 { "id": 8, "name": "Kate" },
 { "id": 9, "name": "Klein" }
 ]
```

- Sastoji se od popisa korisnika, od kojih je svaki predstavljen rječnikom koji za svakog korisnika sadrži njegov ili njezin ID (što je broj) i ime (koje se, u jednoj od velikih kozmičkih slučajnosti, rimuje s ID-om korisnika).
- Također vam daje podatke o „prijateljstvu“, predstavljene kao popis parova ID-ova:

friendships = [(0, 1), (0, 2), (1, 2), (1, 3), (2, 3), (3, 4), (4, 5), (5, 6), (5, 7), (6, 8), (7, 8), (8, 9)]

Pronalaženje ključnih konektora

- Na primjer, n-torka (0, 1) označava da su data scientist-i s ID-om 0 (*Hero*) i data scientist s ID-om 1 (*Dunn*) prijatelji.
- Mreža je ilustrirana na



- Budući da smo korisnike predstavili kao rječnike, lako ih je nadopuniti dodatnim podacima.

Pronalaženje ključnih konektora

- Na primjer, možda bismo htjeli dodati popis prijatelja svakom korisniku.
- Prvo postavljamo svojstvo prijatelja svakog korisnika na prazan popis:

```
for user in users:
    user["friends"] = []
```

- A zatim popise popunjavamo koristeći podatke o prijateljstvima:

```
for i, j in friendships:
# this works because users[i] is the user whose id is i
    users[i]["friends"].append(users[j]) # add i as a friend of j
    users[j]["friends"].append(users[i]) # add j as a friend of i
```

Pronalaženje ključnih konektora

- Nakon što svaki korisnički rječnik sadrži popis prijatelja, lako možemo postavljati pitanja našem grafu, poput **„koji je prosječan broj veza?“**
- Prvo pronalazimo ukupan broj veza zbrajanjem dužina svih popisa prijatelja:

```
def number_of_friends(user):
    """how many friends does _user_ have?"""
    return len(user["friends"])                    # length of friend_ids list

total_connections = sum(number_of_friends(user)
                        for user in users)         # 24
```

- A onda samo podijelimo s brojem korisnika:

```
from __future__ import division                    # integer division is lame
num_users = len(users)                             # length of the users list
avg_connections = total_connections / num_users    # 2.4
```

Pronalaženje ključnih konektora

- Također je lako pronaći najpovezanije ljude - to su ljudi koji imaju najveći broj prijatelja.
- Budući da nema puno korisnika, možemo ih sortirati od "najviše prijatelja" do "najmanje prijatelja":

```
# create a list (user_id, number_of_friends)
num_friends_by_id = [(user["id"], number_of_friends(user))
                     for user in users]

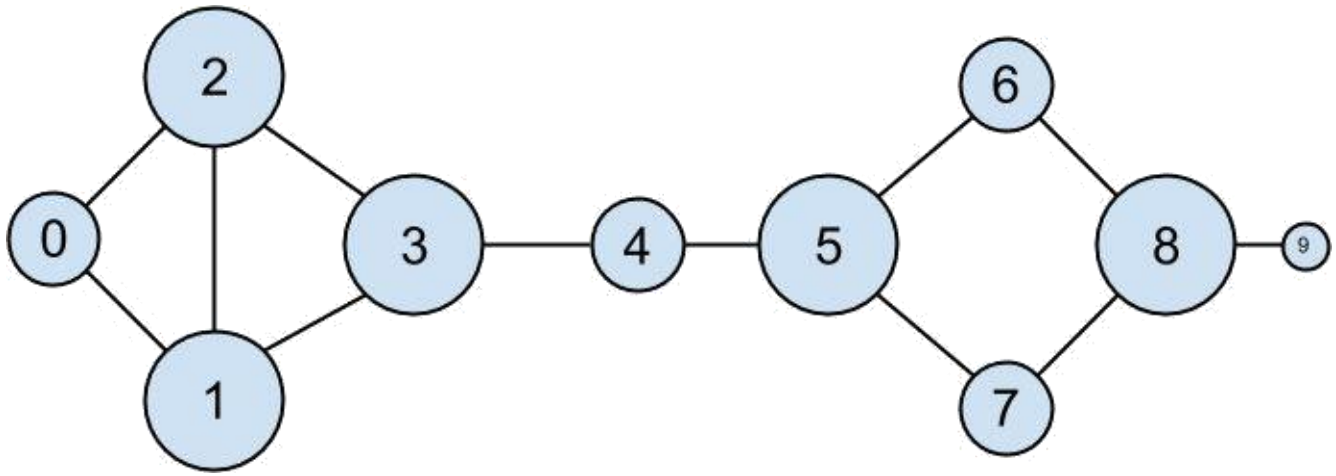
sorted(num_friends_by_id,
       key=lambda (user_id, num_friends): num_friends,
       reverse=True)

# each pair is (user_id, num_friends)
# [(1, 3), (2, 3), (3, 3), (5, 3), (8, 3),
# (0, 2), (4, 2), (6, 2), (7, 2), (9, 1)]
```

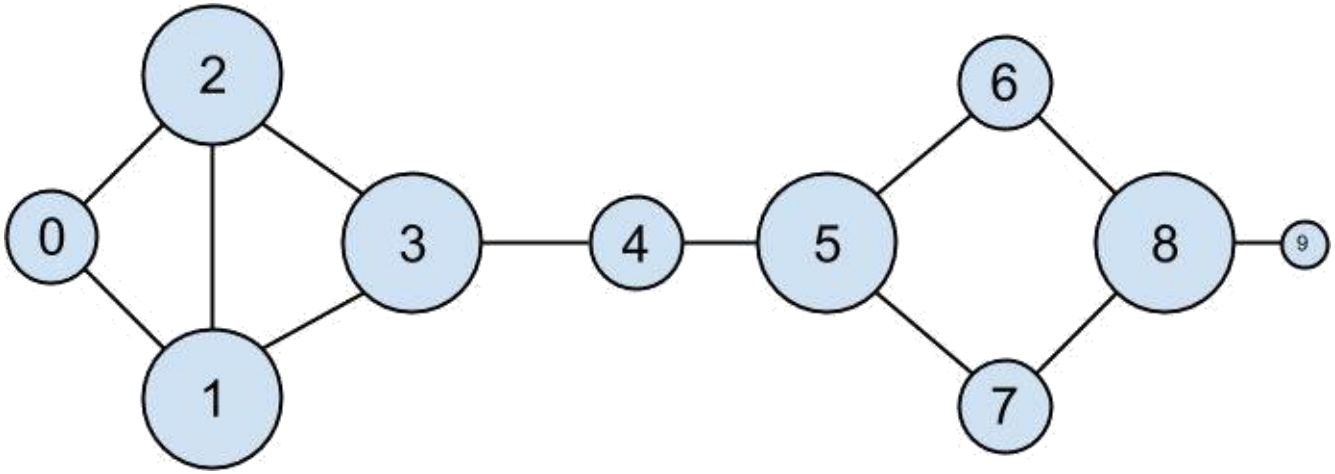
```
# get it sorted
# by num_friends
# largest to smallest
```

Pronalaženje ključnih konektora

- Jedan od načina da se shvati ono što smo učinili jest da se identificiraju ljudi koji su na neki način središnji dio mreže.
- Zapravo, ono što smo upravo izračunali je stupanj centralnosti metrike mreže.



Pronalaženje ključnih konektora



- Prednost ovoga je što se prilično lako izračunava, ali ne daje uvijek rezultate koje biste željeli ili očekivali.
- Na primjer, u DataSciencester mreži *Thor* (id 4) ima samo dvije veze, dok *Dunn* (id 1) ima tri.
- Ipak, gledajući mrežu, intuitivno se čini da bi *Thor* trebao biti centralniji.

Plate i iskustvo

- Baš kad se spremate za pauzu, VP za HR pita Vas možete li navesti neke zanimljive činjenice o tome koliko data scientist-i zarađuju.
- Podaci o platama su naravno osjetljivi, ali uspijeva Vam pružiti anonimni skup podataka koji sadrži platu svakog korisnika (u dolarima) i radni staž kao data scientist (u godinama):

```
salaries_and_tenures = [(83000, 8.7), (88000, 8.1),
                        (48000, 0.7), (76000, 6),
                        (69000, 6.5), (76000, 7.5),
                        (60000, 2.5), (83000, 10),
                        (48000, 1.9), (63000, 4.2)]
```

Plate i iskustvo

- Prirodan prvi korak je grafički prikaz podataka.
- Rezultate možete vidjeti

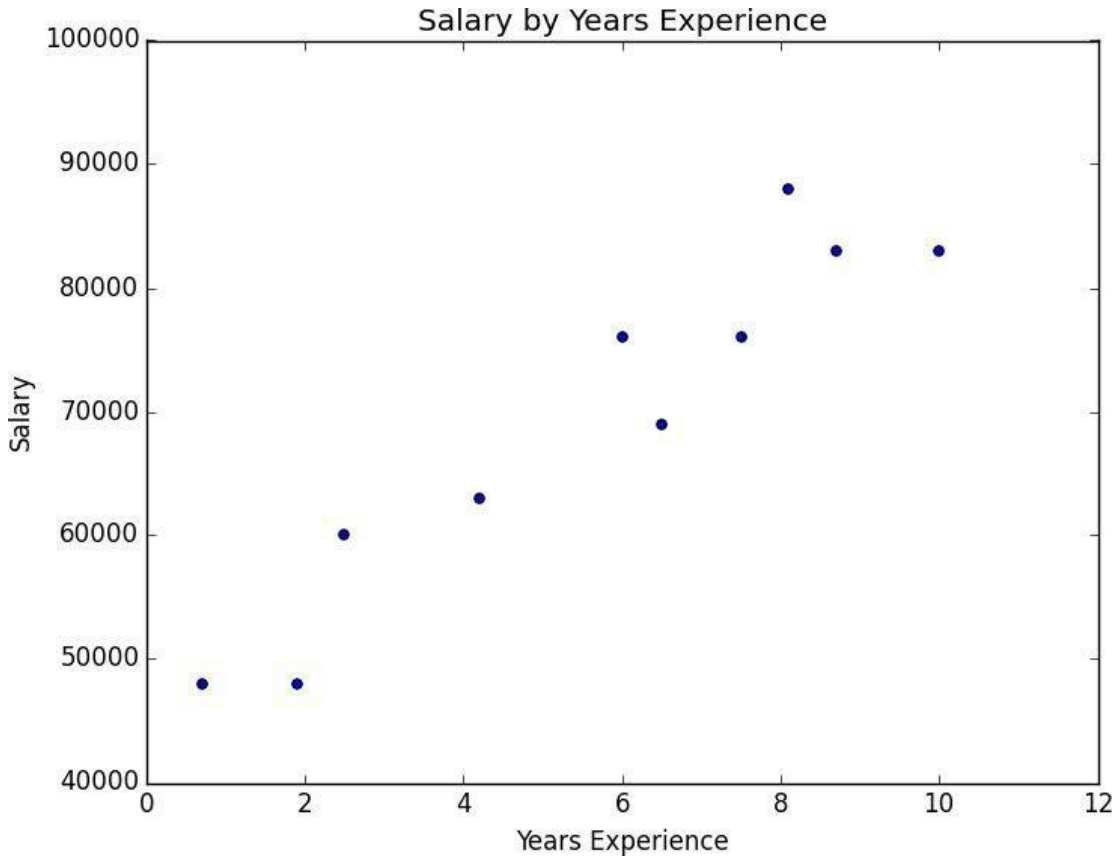


Plate i iskustvo

- Čini se prilično jasnim da ljudi s više iskustva obično zarađuju više.
- Kako to možete pretvoriti u zabavnu činjenicu? Vaša prva ideja je pogledati prosječnu platu za svaki staž:

```
#   keys are years, values are lists of the salaries for each tenure
    salary_by_tenure = defaultdict(list)

            for salary, tenure in salaries_and_tenures:
              salary_by_tenure[tenure].append(salary)


#   keys are years, each value is average salary for that tenure
    average_salary_by_tenure = {
      tenure : sum(salaries) / len(salaries)
      for tenure, salaries in salary_by_tenure.items()
  }
```

Plate i iskustvo

- To se ispostavilo kao ne baš korisno, jer nijedan od korisnika nema isti staž, što znači da samo izvještavamo o platama pojedinačnih korisnika:

```
{0.7: 48000.0,
 1.9: 48000.0,
   2.5: 60000.0,
   4.2: 63000.0,
6.   76000.0,
     6.5:
     69000.0,
   7.5: 76000.0,
   8.1: 88000.0,
   8.7: 83000.0,
10. 83000.0}
```

Plate i iskustvo

- Možda bi bilo korisnije grupirati mandate:

```
def tenure_bucket(tenure):
    if tenure < 2:
        return "less than two"
    elif tenure < 5:
        return "between two and five"
    else:
        return "more than five"
```

- Zatim grupirajte plaće koje odgovaraju svakoj skupini:

```
#    keys are tenure buckets, values are lists of salaries for that
     bucket salary_by_tenure_bucket = defaultdict(list)

for salary, tenure in salaries_and_tenures:
    bucket = tenure_bucket(tenure)
    salary_by_tenure_bucket[bucket].append(salary)
```

Plate i iskustvo

- I na kraju izračunajte prosječnu plaću za svaku skupinu:

```
#    keys are tenure buckets, values are average
     salary for that bucket
     average_salary_by_bucket = {
    tenure_bucket : sum(salaries) / len(salaries)
    for tenure_bucket, salaries in salary_by_tenure_bucket.iteritems()
  }
```

- što je zanimljivije:

```
{'between two and five': 61500.0,
 'less than two': 48000.0,
 'more than five': 79166.66666666667}
```

Plate i iskustvo

- I imate svoj zvučni paragraf:
- *„Data Scientist-i s više od pet godina iskustva zarađuju 65% više od znanstvenika podataka s malo ili bez iskustva!“*
- Ali mi smo odabrali kategorije na prilično proizvoljan način.
- Ono što bismo zapravo željeli jest dati neku vrstu izjave o utjecaju plate - u prosjeku - dodatne godine iskustva.
- Osim što je to zanimljivija činjenica, ovo nam omogućuje da dajemo predviđanja o platama koje ne znamo.

Plaćeni računi

- Kad se vratite za svoj stol, VP za prihode Vas čeka.
- Želi bolje razumjeti koji korisnici plaćaju za račune, a koji ne. (Zna njihova imena, ali to nije osobito korisna informacija.)
- Primjećujete da postoji podudarnost između godina iskustva i plaćenih računa:

0.7	paid
1.9	unpaid
2.5	paid
4.2	unpaid
6	unpaid
6.5	unpaid
7.5	unpaid
8.1	unpaid
8.7	paid
10	paid

- Korisnici s vrlo malo i vrlo puno godina iskustva obično plaćaju; korisnici s prosječnim iskustvom ne.

Plaćeni računi

- Sukladno tome, ako želite stvoriti model, iako to definitivno nije dovoljno podataka za izradu modela, mogli biste pokušati predvidjeti "plaćeno" za korisnike s vrlo malo i vrlo puno godina iskustva, te "neplaćeno" za korisnike s osrednjom količinom iskustva:

```
def
predict_paid_or_unpaid(years_experience):
  if years_experience < 3.0:
    return "paid"
  elif years_experience < 8.5:
    return "unpaid"
  else:
    return "paid"
```

- Naravno, potpuno smo procijenili granice. S više podataka (i više matematike), mogli bismo izgraditi model koji predviđa vjerovatnost da će korisnik platiti, na temelju njegovih godina iskustva.

Teme od interesa

- Dok završavate svoj prvi dan, VP za strategiju sadržaja traži od vas podatke o temama koje najviše zanimaju korisnike kako bi mogla u skladu s tim isplanirati svoj blog kalendar. Već imate sirove podatke iz projekta predlaganja prijatelja:

```
interests = [
    (0, "Hadoop"), (0, "Big Data"), (0, "HBase"), (0, "Java"), (0,
    "Spark"), (0, "Storm"), (0, "Cassandra"),
    (1, "NoSQL"), (1, "MongoDB"), (1, "Cassandra"), (1, "HBase"),
    (1, "Postgres"), (2, "Python"), (2, "scikit-learn"), (2, "scipy"),
    (2, "numpy"), (2, "statsmodels"), (2, "pandas"), (3, "R"), (3, "Python"), (3,
    "statistics"), (3, "regression"), (3, "probability"),
    (4, "machine learning"), (4, "regression"), (4, "decision trees"), (4,
    "libsvm"), (5, "Python"), (5, "R"), (5, "Java"), (5, "C++"), (5, "Haskell"), (5,
    "programming languages"), (6, "statistics"), (6, "probability"), (6,
    "mathematics"), (6, "theory"),

    (7, "machine learning"), (7, "scikit-learn"), (7, "Mahout"),
    (7, "neural networks"), (8, "neural networks"), (8, "deep learning"), (8, "Big
    Data"), (8, "artificial intelligence"), (9, "Hadoop"), (9, "Java"), (9,
    "MapReduce"), (9, "Big Data")

]
```

Teme od interesa

- Jedan jednostavan (iako ne i osobito uzbudljiv) način pronalaženja najpopularnijih interesa je jednostavno brojanje riječi:
  1. Napišite svaki interes malim slovom (budući da različiti korisnici mogu, ali i ne moraju, pisati svoje interese velikim slovom).
  2. Podijelite ga na riječi.
  3. Prebrojite rezultate.
- U kodu:

```
words_and_counts = Counter(word
                           for user, interest in interests
for word in interest.lower().split())
```

Teme od interesa

- Ovo olakšava nabrajanje riječi koje se pojavljuju više puta:

```
for word, count in
words_and_counts.most_common():
    if count > 1:
print word, count
```

- što daje rezultate koje biste očekivali (osim ako očekujete da će se "scikit-learn" podijeliti na dvije riječi, u kojem slučaju ne daje rezultate koje očekujete):

learning 3	statistics 2
java 3	probability 2
python 3	hadoop 2
big 3	networks 2
data 3	machine 2
hbase 2	neural 2
regression 2	scikit-learn 2
cassandra 2	r 2

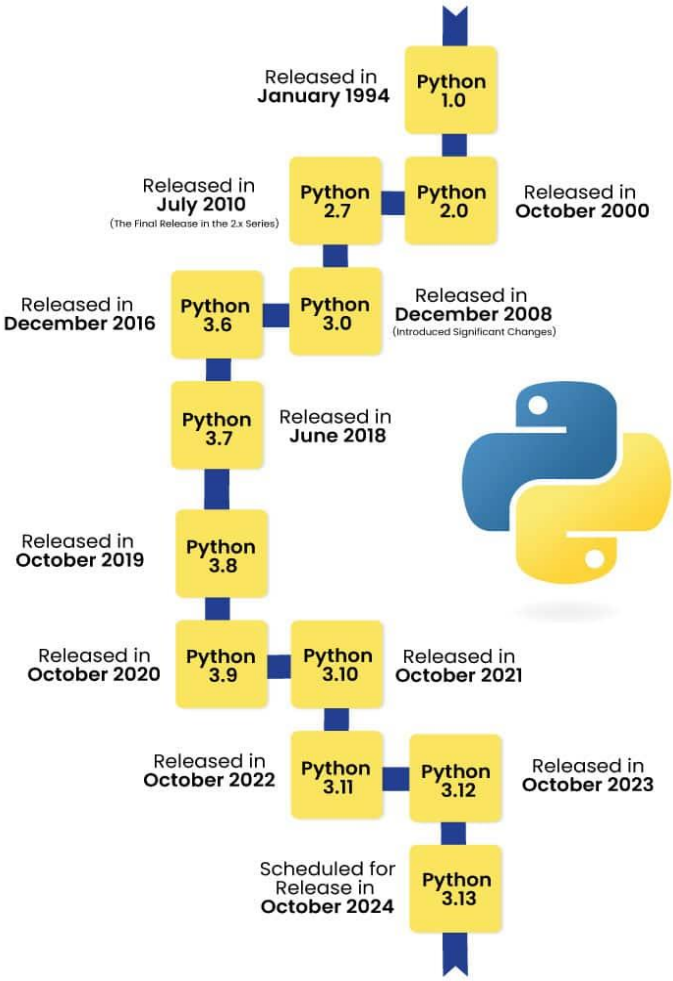
Python

- *Ljudi su i nakon dvadeset pet godina još uvijek ludi za Pythonom, što mi je teško povjerovati.* Michael Palin, 2015
- Python je stvorio **Guido van Rossum** krajem 1980-ih, a prvo službeno izdanje code-a (verzija 0.9.0) bilo je u februaru 1991.
- Početkom 2026., Python ima 35 godina.
- Zamišljen je kao nasljednik programskog jezika ABC i evoluirao je u jedan od najpopularnijih programskih jezika.



Python

- Python-ova evolucija 1994:
  S lansiranjem Pythona 1.0 uvedene su značajke poput lambde, mapiranja, filtriranja i smanjenja.
- 2000: S lansiranjem Pythona 2.0 uvedeno je razumijevanje lista i napredni sustav sakupljanja smeća.
- 2008: S lansiranjem Pythona 3.0 („Py3K“) rješavaju se ključne slabosti jezika.



Key Features of Python

- **Jednostavan za učenje:** Python je izvrstan za početnike bez potrebe za predznanjem.
- **High-Level Language:** Nema potrebe za brigom o detaljima niske razine poput upravljanja memorijom.
- **Interpretiran:** Izvršava kod redak po redak, što omogućuje:
  - Izvođenje koda na različitim platformama.
  - Promjene se mogu napraviti u hodu bez potrebe za ponovnim pokretanjem programa.
- **Cross-Platform:** Besprijekorno radi na Linuxu i macOS-u bez potrebe za ikakvim modifikacijama sustava Windows.
- **Dynamically Typed:** Tipovi varijabli provjeravaju se za vrijeme izvođenja (dinamički), eliminirajući potrebu za njihovim deklariranjem.

Key Features of Python

- **Objektno orijentiran:** Pruža podršku za OOP koncepte kao što su klase, nasljeđivanje i polimorfizam, što rezultira modularnim, višekratno upotrebljivim i održivim kodom.
- **Opsežne biblioteke:** Dolazi s velikim skupom biblioteka i modula za brži razvoj.
- **Open-Source:** Besplatan za korištenje s velikom zajednicom koja doprinosi njegovom rastu.
- **High Demend:** Popularan u industrijama poput umjetne inteligencije, razvoja softvera, financija i računarstva u oblaku.

Korištenje Pythona

- Python nudi niz biblioteka i framework-a te je programski jezik visoke razine open-source-e code-a. Python je stekao popularnost zbog svoje jednostavnosti, jednostavne sintakse i korisničkog okruženja

Technology	Python Libraries
Desktop Applications	Tkinter, PyQt, Kivy, wxPython
Web Applications	Django, Flask, FastAPI, Pyramid
Data Science	NumPy, Pandas, SciPy, Matplotlib, Seaborn
Artificial Intelligence	TensorFlow, Keras, PyTorch, OpenAI Gym, Hugging Face Transformers
Machine Learning	Scikit-learn, XGBoost, LightGBM, CatBoost, Statsmodels
Scientific Computing	SciPy, SymPy, NumPy, Matplotlib, Pandas
Robotics	ROS (Robot Operating System), Pypot, PyRobot, VEX Robotics
Internet of Things (IoT)	Raspberry Pi libraries, MicroPython, CircuitPython
Gaming	Pygame, Panda3D, Arcade, PyKyra
Mobile Apps	Kivy, BeeWare, PyQt, PySide
Data Analysis and Preprocessing	Pandas, NumPy, SciPy, Dask, Vaex, OpenCV

Korištenje Pythona

- Python nudi niz biblioteka i framework-a te je programski jezik visoke razine open-source-e code-a. Python je stekao popularnost zbog svoje jednostavnosti, jednostavne sintakse i korisničkog okruženja

Technology	Python Libraries
Desktop Applications	Tkinter, PyQt, Kivy, wxPython
Web Applications	Django, Flask, FastAPI, Pyramid
Data Science	NumPy, Pandas, SciPy, Matplotlib, Seaborn
Artificial Intelligence	TensorFlow, Keras, PyTorch, OpenAI Gym, Hugging Face Transformers
Machine Learning	Scikit-learn, XGBoost, LightGBM, CatBoost, Statsmodels
Scientific Computing	SciPy, SymPy, NumPy, Matplotlib, Pandas
Robotics	ROS (Robot Operating System), Pypot, PyRobot, VEX Robotics
Internet of Things (IoT)	Raspberry Pi libraries, MicroPython, CircuitPython
Gaming	Pygame, Panda3D, Arcade, PyKyra
Mobile Apps	Kivy, BeeWare, PyQt, PySide
Data Analysis and Preprocessing	Pandas, NumPy, SciPy, Dask, Vaex, OpenCV

Python

Formatiranje praznih prostora

- Mnogi jezici koriste vitičaste zagrade za razgraničenje blokova code-a. Python koristi uvlačenje:

```
for i in [1, 2, 3, 4, 5]:
    print i
    for j in [1, 2, 3, 4, 5]:
        print j
        print i + j
    print i
print "done looping"
```

- Zbog toga je Python kod vrlo čitljiv, ali također znači da morate biti vrlo oprezni s formatiranjem. Razmaci se zanemaruju unutar zagrada, što može biti korisno za dugačke izračune:

```
long_winded_computation = (1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10
                           + 11 + 12 + 13+14+15+16+17+18+19+20)
```

Python
Formatiranje praznih prostora

- i za lakše čitanje code-a:

```
list_of_lists = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

easier_to_read_list_of_lists = [ [1, 2, 3],
                                 [4, 5, 6],
                                 [7, 8, 9] ]
```

- Također možete koristiti obrnutu kosu crtu kako biste naznačili da se naredba nastavlja u sljedećem retku, iako ćemo to rijetko raditi:

```
two_plus_three = 2 + \
3
```

- Jedna od posljedica formatiranja praznih prostora je ta što može biti teško kopirati i zalijepiti code u Python shell. Na primjer, ako ste pokušali zalijepiti code: `for i in [1, 2, 3, 4, 5]:  #notice the blank line print i`
- u običnu Python shell, dobili biste: **IndentationError**: expected an indented block
- jer interpreter misli da prazan redak signalizira kraj bloka for petlje.
- Python ima čarobnu funkciju **%paste** koja ispravno lijepi sve što se nalazi u Vašem međuspremniku, uključujući i prazne prostore.

Python
Moduli

- Određene značajke Pythona ne učitavaju se prema default postavkama. To uključuje feature uključene kao dio jezika, kao i feature trećih strana koje sami downloadirate. Da biste koristili te feature, morat ćete uvesti module koji ih sadrže. Jedan od pristupa je jednostavno uvesti sam modul:

```
import re
my_regex = re.compile("[0-9]+", re.I)
```

- Ovdje je **re** modul koji sadrži funkcije i konstante za rad s regularnim izrazima. Nakon ove vrste **import**-a tim funkcijama možete pristupiti samo dodavanjem prefiksa **re**..
- Ako ste već imali drugačiji re u svom kodu, mogli biste koristiti alias:

```
import re as regex
my_regex = regex.compile("[0-9]+", regex.I)
```

Python
Moduli

- To možete učiniti i ako Vaš modul ima nezgrapno ime ili ako ćete ga puno tipkati. Na primjer, prilikom vizualizacije podataka pomoću matplotliba, standardna konvencija je:

```
import matplotlib.pyplot as plt
```

- Ako Vam je potrebno nekoliko specifičnih vrijednosti iz modula, možete ih eksplicitno import-irati i koristiti bez kvalifikacija:

```
from collections import defaultdict, Counter
lookup = defaultdict(int)
my_counter = Counter()
```

- Da ste loša osoba, mogli biste uvesti cijeli sadržaj modula u svoj namespace, što bi moglo nenamjerno prebrisati varijable koje ste već definirali:

```
match = 10
from re import *    # uh oh, re has a match function
print match      # "<function re.match>"
```

- Međutim, budući da niste loša osoba, to nikada nećete učiniti.

Python
Aritmetika

- Python 2.7 koristi cjelobrojno dijeljenje prema zadanim postavkama, tako da je **5 / 2** jednako **2**. Gotovo uvijek to nije ono što želimo, pa ćemo uvijek započeti naše datoteke s:

```
from __future__ import division
```

- nakon čega je **5/2** jednako **2,5**.
- Gotovo svaki primjer code-a u koristi ovo dijeljenje novog stila.
- U nekoliko slučajeva kada nam je potrebno cjelobrojno dijeljenje, možemo ga dobiti dvostrukom kosom crtom: **5 // 2**.

Python
Funkcije

- Funkcija je pravilo za uzimanje nula ili više ulaza i vraćanje odgovarajućeg izlaza. U Pythonu funkcije obično definiramo pomoću def:

```
def double(x):
    """this is where you put an optional docstring
    that explains what the function does.
    for example, this function multiplies its input by 2"""
    return x * 2
```

- Python funkcije su prvoklasne, što znači da ih možemo dodijeliti varijablama i predati funkcijama baš kao i bilo koje druge argumente:

```
def apply_to_one(f):
    """calls the function f with 1 as its argument"""
    return f(1)

my_double = double      # refers to the previously defined function
x = apply_to_one(my_double) # equals 2
```

- Također je jednostavno kreirati kratke anonimne funkcije ili lambde:

```
y = apply_to_one(lambda x: x + 4)        # equals 5
```

Python
Funkcije

- Lambde možete dodijeliti varijablama, iako će vam većina ljudi reći da biste umjesto toga trebali koristiti **def**:

```
another_double = lambda x: 2 * x          # don't do this
def another_double(x): return 2 * x       # do this instead
```

- Parametrima funkcija mogu se dodijeliti i zadani argumenti, koje je potrebno navesti samo kada želite vrijednost koja nije zadana:

```
def my_print(message="my default message"):
    print message

my_print("hello")    # prints 'hello'
my_print()          # prints 'my default message'
```

- Ponekad je korisno navesti argumente po imenu:

```
def subtract(a=0, b=0):
    return a - b

subtract(10, 5) # returns 5
subtract(0, 5)        # returns -5
subtract(b=5)         # same as previous
```

- Stvorit ćemo mnogo, mnogo funkcija.

Python
Stringovi (Nizovi)

- Nizovi mogu biti odvojeni jednostrukim ili dvostrukim navodnicima (ali navodnici se moraju podudarati):

```
single_quoted_string = 'data science'
double_quoted_string = "data science"
```

- Python koristi obrnute kose crte za kodiranje posebnih znakova. Na primjer:

```
tab_string = "\t"      # represents the tab character
len(tab_string)        # is 1
```

- Ako želite obrnute kose crte kao obrnute kose crte (što možete vidjeti u nazivima direktorija sustava Windows ili u regularnim izrazima), možete stvoriti sirove nizove pomoću r"":

```
not_tab_string = r"\t" # represents the characters '\' and 't'
len(not_tab_string)            # is 2
```

- Možete stvoriti višelinijske nizove pomoću trostrukih-[dvostrukih-]-navodnika:

```
multi_line_string = """This is the first line.
and this is the second line
and this is the third line"""
```

Python
Exceptions (Iznimke)

- Kada nešto krene po zlu, Python generira iznimku. Ako se ne obrade, one će uzrokovati rušenje programa. Možete ih obraditi pomoću **try** i **except**:

```
try:
    print 0 / 0
except ZeroDivisionError:
    print "cannot divide by zero"
```

- Iako se u mnogim jezicima iznimke smatraju lošima, u Pythonu nema sramote koristiti ih za čišćenje code-a i povremeno ćemo to činiti.

Python
Lists (liste, popisi)

- Vjerojatno najosnovnija struktura podataka u Pythonu je popis.
- Popis je jednostavno uređena kolekcija. (Slično je onome što se u drugim jezicima naziva nizom, ali s nekim dodatnim funkcijama.)

```
integer_list = [1, 2, 3]
heterogeneous_list = ["string", 0.1, True]
list_of_lists = [ integer_list, heterogeneous_list, [] ]

list_length =       len(integer_list)     #          equals     3
list_sum    =       sum(integer_list)     #          equals     6
```

- N-ti element liste možete dobiti ili postaviti pomoću uglatih zagrada:

```
x = range(10)          # is the list [0, 1, ..., 9]
zero = x[0]            # equals 0, lists are 0-indexed
one = x[1]             # equals     1
nine = x[-1]           # equals     9, 'Pythonic' for last element
eight = x[-2]          # equals 8, 'Pythonic' for next-to-last element
x[0] = -1              # now x is [-1, 1, 2, 3, ..., 9]
```

Python
Lists (liste, popisi)

- Također možete koristiti uglate zagrade za "rezanje" popisa:

```
first_three   = x[:3]                  # [-1, 1, 2]
three_to_end = x[3:]                   # [3, 4, ..., 9]
one_to_four = x[1:5]                   # [1, 2, 3, 4]
last_three = x[-3:]                    # [7, 8, 9]
without_first_and_last = x[1:-1]       # [1, 2, ..., 8]
copy_of_x = x[:]                       # [-1, 1, 2, ..., 9]
```

- Python ima operator in za provjeru članstva u listi:

```
1 in    [1,   2,   3]     # True
0 in    [1,   2,   3]     # False
```

- Ova provjera uključuje ispitivanje elemenata popisa jedan po jedan, što znači da ga vjerojatno ne biste trebali koristiti osim ako znate da je vaš popis prilično malen (ili osim ako vam nije važno koliko će provjera trajati).

Python
Lists (liste, popisi)

- Jednostavno je spajanje popisa:

```
x = [1, 2, 3]
x.extend([4, 5, 6])  # x is now [1,2,3,4,5,6]
```

- Ako ne želite mijenjati **x**, možete koristiti zbrajanje popisa:

```
x = [1, 2, 3]
y = x + [4, 5, 6]    # y is [1, 2, 3, 4, 5, 6]; x is unchanged
```

- Češće ćemo popisima dodavati jedan po jedan element:

```
x = [1, 2, 3]
x.append(0)          # x is now [1, 2, 3, 0]
y = x[-1]            # equals    0
z = len(x)           # equals    4
```

Python
Lists (liste, popisi)

- Često je praktično ***unpack*** popise ako znate koliko elemenata sadrže:

```
x, y = [1, 2]        # now x is 1, y is 2
```

- iako ćete dobiti ***ValueError*** ako nemate isti broj elemenata s obje strane.
- Uobičajeno je koristiti podvlaku za vrijednost koju ćete odbaciti:

```
_, y = [1, 2]        # now y == 2, didn't care about the first element
```

Python
Ntorke



- Ntorke su nepromjenjivi rođaci lista.
- Gotovo sve što možete učiniti s listom, a da ne uključuje njezino mijenjanje, možete učiniti i s ntorkom.
- Ntorku određujete korištenjem zagrada (ili ničega) umjesto uglatih zagrada:

```
my_list = [1, 2]
my_tuple = (1, 2)
other_tuple = 3, 4
my_list[1] = 3    # my_list is now [1, 3]

try:
    my_tuple[1] = 3
except TypeError:
    print "cannot modify a tuple"
```

Python
Ntorke

- Ntorke su prikladan način za vraćanje više vrijednosti iz funkcija:

```
def sum_and_product(x, y):
    return (x + y),(x * y)

sp   =  sum_and_product(2, 3)          # equals (5,      6)
s,   p          = sum_and_product(5, 10) # s is 15, p      is   50
```

- Ntorke (i liste) se također mogu koristiti za višestruko dodjeljivanje:

```
x, y = 1, 2     # now x is 1, y is 2
x, y = y, x     # Pythonic way to swap variables; now x is 2, y is 1
```

Python
Rječnici

- Još jedna temeljna struktura podataka je rječnik koji povezuje vrijednosti s ključevima i omogućuje Vam brzo pronalaženje vrijednosti koja odgovara zadanom ključu:

```
empty_dict = {}                                    # Pythonic
empty_dict2 = dict()                    # less Pythonic
grades = { "Joel" : 80, "Tim" : 95 }# dictionary literal
```

- Vrijednost ključa možete potražiti pomoću uglatih zagrada:

```
joels_grade = grades["Joel"]        # equals 80
```

- Ali dobit ćete KeyError ako zatražite ključ koji nije u rječniku:

```
try:
    kates_grade = grades["Kate"]
except KeyError:
    print "no grade for Kate!"
```

Python
Rječnici

- Postojanje ključa možete provjeriti pomoću:

```
joel_has_grade = "Joel" in grades          # True
kate_has_grade = "Kate" in grades          # False
```

- Rječnici imaju metodu get koja vraća zadanu vrijednost (umjesto iznimke) kada tražite ključ koji nije u rječniku:

```
joels_grade = grades.get("Joel",          0)   # equals    80
kates_grade = grades.get("Kate",          0)   # equals    0
no_ones_grade = grades.get("No One")           # default default is None
```

- Parove ključ-vrijednost dodjeljujete pomoću istih uglatih zagrada:

```
grades["Tim"] = 99                 # replaces the old value
grades["Kate"] = 100               # adds a third entry
num_students = len(grades)         # equals 3
```

Python
Rječnici

- Često ćemo koristiti rječnike kao jednostavan način predstavljanja strukturiranih podataka:

```
tweet = {
    "user" : "joelgrus",
    "text" : "Data Science is Awesome",
    "retweet_count" : 100,
    "hashtags" : ["#data", "#science", "#datascience",
    "#awesome", "#yolo"]
}
```

- Osim traženja specifičnih ključeva, možemo ih pogledati sve:

```
tweet_keys      = tweet.keys()      # list of keys
tweet_values    = tweet.values()    # list of values
tweet_items     = tweet.items()     # list of (key, value) tuples

"user" in tweet_keys                # True, but uses a slow list in
"user" in tweet                     # more Pythonic, uses faster dict in
"joelgrus" in tweet_values          # True
```

- Ključevi rječnika moraju biti nepromjenjivi; posebno, ne možete koristiti popise kao ključeve. Ako vam je potreban višedijelni ključ, trebali biste koristiti tuple ili smisliti način da ključ pretvorite u niz.

Python
Rječnici - **defaultdict**

- Zamislite da pokušavate prebrojati riječi u dokumentu. Očiti pristup je stvaranje rječnika u kojem su ključevi riječi, a vrijednosti brojevi. Dok provjeravate svaku riječ, možete povećati njezin broj ako je već u rječniku i dodati je u rječnik ako nije:

```
word_counts = {}
for word in document:
    if word in word_counts:
        word_counts[word] += 1
    else:
        word_counts[word] = 1
```

- Također možete koristiti pristup **"oprost je bolji od dopuštenja"** i samo obraditi iznimku od pokušaja traženja nedostajućeg ključa:

```
word_counts = {}
for word in document:
    try:
        word_counts[word] += 1
    except KeyError:
        word_counts[word] = 1
```

Python
Rječnici - **defaultdict**

- Treći pristup je korištenje funkcije **get**, koja se elegantno ponaša za nedostajuće ključeve:

```
word_counts = {}
for word in document:
    previous_count = word_counts.get(word, 0)
    word_counts[word] = previous_count + 1
```

- Svaki od ovih je pomalo nezgrapan, zbog čega je **defaultdict** koristan. Defaultdict je poput običnog rječnika, osim što kada pokušate pronaći ključ koji ne sadrži, prvo mu dodaje vrijednost koristeći funkciju s nula argumenata koju ste naveli prilikom stvaranja. Da biste koristili **defaultdicts**, morate ih uvesti iz kolekcija:

```
from collections import defaultdict

word_counts = defaultdict(int) # int() produces
0 for word in document:

    word_counts[word] += 1
```

Python
Rječnici - **defaultdict**



- Također mogu biti korisni s listama ili dictovima ili čak vašim vlastitim funkcijama:

```
dd_list = defaultdict(list)                                  # list() produces an empty list
dd_list[2].append(1)                                         # now dd_list contains {2: [1]}
dd_dict = defaultdict(dict)                                  # dict() produces an empty dict
dd_dict["Joel"]["City"] = "Seattle“                          # { "Joel" : { "City" : Seattle"}}
dd_pair = defaultdict(lambda: [0, 0]) dd_pair[2][1] = 1      # now dd_pair contains {2: [0,1]}
```

- Ovi će biti korisni kada koristimo rječnike za "prikupljanje" rezultata po nekom ključu i ne želimo svaki put provjeravati postoji li ključ.

Python
*Counter (*Brojač)

- Counter pretvara niz vrijednosti u objekt sličan defaultdict(int) koji mapira ključeve na counts. Primarno ćemo ga koristiti za izradu histograma:

```
from collections import Counter
c = Counter([0, 1, 2, 0])        # c is (basically) { 0 : 2, 1 : 1, 2 : 1 }
```

- To nam daje vrlo jednostavan način rješavanja našeg problema word_counts:

```
word_counts = Counter(document)
```

- Instanca Counter ima metodu most_common koja je često korisna:

```
#    print the 10 most common words and their counts for word,
count in word_counts.most_common(10):
  print word, count
```

Python
*Counter (*Brojač)

- Counter pretvara niz vrijednosti u objekt sličan defaultdict(int) koji mapira ključeve na counts. Primarno ćemo ga koristiti za izradu histograma:

```
from collections import Counter
c = Counter([0, 1, 2, 0])        # c is (basically) { 0 : 2, 1 : 1, 2 : 1 }
```

- To nam daje vrlo jednostavan način rješavanja našeg problema word_counts:

```
word_counts = Counter(document)
```

- Instanca Counter ima metodu most_common koja je često korisna:

```
#    print the 10 most common words and their counts for word,
count in word_counts.most_common(10):
  print word, count
```

Python
Sets (Skupovi)

- Druga struktura podataka je skup, koji predstavlja kolekciju različitih elemenata:

```
s = set()
s.add(1)          # s is now { 1 }
s.add(2)          # s is now { 1, 2 }
s.add(2)          # s is still { 1, 2 }
x = len(s)        # equals 2
y = 2    in s     # equals True
z = 3    in s     # equals False
```

- Skupove ćemo koristiti iz dva glavna razloga. Prvi je taj što je vrlo brza operacija na skupovima. Ako imamo veliku kolekciju elemenata koje želimo koristiti za test pripadnosti, skup je prikladniji od popisa:

```
stopwords_list = ["a","an","at"] + hundreds_of_other_words + ["yet", "you"]

"zip" in stopwords_list  # False, but have to check every element

stopwords_set = set(stopwords_list)
"zip" in stopwords_set  # very fast to check
```

Python
Sets (Skupovi)

- Drugi razlog je pronaći različite elemente u kolekciji:

```
item_list = [1, 2, 3, 1, 2, 3]
num_items = len(item_list)                  # 6
item_set = set(item_list)                   # {1, 2, 3}
num_distinct_items = len(item_set)          #  3
distinct_item_list = list(item_set)         #  [1, 2, 3]
```

- Skupove ćemo koristiti mnogo rjeđe nego rječnike i liste.

Python
*Control Flow (*Kontrolni tok)

- Kao i u većini programskih jezika, radnju možete uvjetno izvršiti pomoću if:

```
if 1 > 2:
    message = "if only 1 were greater than two…"
elif 1 > 3:
    message = "elif stands for 'else if'"
else:
    message = "when all else fails use else (if you want to)"
```

- Također možete napisati ternarni if-then-else u jednom retku, što ćemo povremeno raditi:

```
parity = "even" if x % 2 == 0 else "odd"
```

- Python ima **while** petlju:

```
x = 0
while x < 10:
    print x, "is less than 10"
    x += 1
```

Python
*Control Flow (*Kontrolni tok)

- iako ćemo češće koristiti **for** i **in**:

```
for x in range(10):
    print x, "is less than 10"
```

- Ako vam je potrebna složenija logika, možete koristiti **continue** i **break**:

```
for x in range(10):
    if x == 3:
        continue        # go immediately to the next
        iteration
    if x == 5:
        break    # quit the loop entirely
    print x
```

- Ovo će ispisati 0, 1, 2 i 4.

Python
Istinitost

- **Boolean** vrijednosti u Pythonu funkcioniraju kao u većini drugih jezika, osim što se pišu velikim slovom:

```
one_is_less_than_two = 1 < 2 # equals True
true_equals_false = True == False           # equals False
```

- Python koristi vrijednost **None** za označavanje nepostojeće vrijednosti. Slično je drugim jezicima **null**:

```
x = None
print x == None       # prints True, but is not Pythonic
print x is None # prints True, and is Pythonic
```

Python
Istinitost

- Python Vam omogućuje korištenje bilo koje vrijednosti tamo gdje očekuje logičku vrijednost. Sljedeće vrijednosti su sve "netačne":
  - False
  - None
  - [] (an empty list)
  - {} (an empty dict)
  - ""
  - set()
  - 0
  - 0.0

Python
Istinitost

- Gotovo sve ostalo se tretira kao **True**. To vam omogućuje jednostavno korištenje **if** naredbi za testiranje praznih popisa, praznih nizova znakova, praznih rječnika ili tako dalje. Također ponekad uzrokuje nezgodne greške ako ne očekujete ovo ponašanje:

```
s = some_function_that_returns_a_string()
if s:
    first_char = s[0]
else:
first_char = ""
```

- Jednostavniji način da se to učini je:

```
first_char = s and s[0]
```

Python
Istinitost

- budući da i vraća svoju drugu vrijednost kada je prva "istinita", prvu vrijednost kada nije. Slično, ako je x broj ili eventualno **None**:

```
safe_x = x or 0
```

- je definitivno broj.
- Python ima funkciju all, koja uzima listu i vraća **True** tačno kada je svaki element istinit, i funkciju **any**, koja vraća **True** kada je barem jedan element istinit:

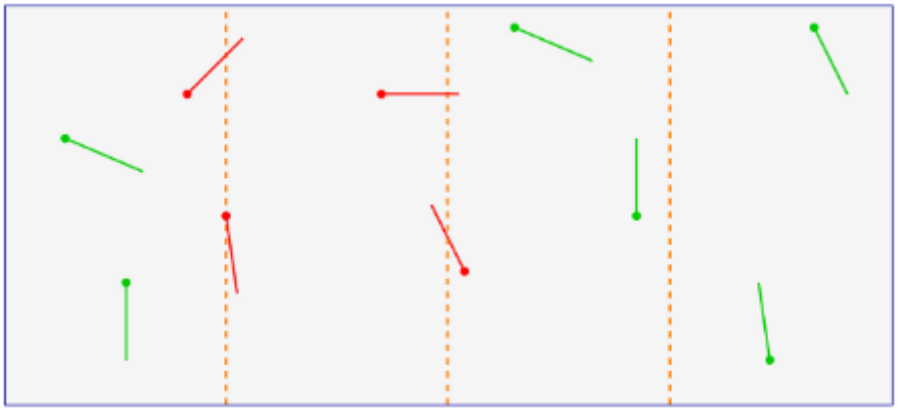
```
all([True, 1, { 3 }])      # True
all([True, 1, {}])         # False, {} is falsy
any([True, 1, {}])         # True, True is truthy
all([])                    # True, no falsy elements in the list
any([])                    # False, no truthy elements in the list
```

Vježba – IN CLASS

- Birthday Paradox

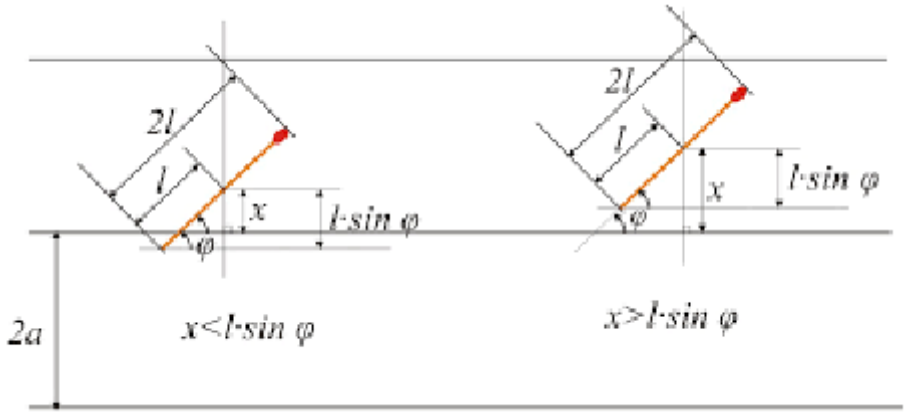
Vježba – Birthday Paradox

- **Početci Monte Carlo metoda**
- Osnovni koncept Monte Carlo metoda potiče još iz 18. stoljeća kada je francuski znanstvenik Buffon prezentirao metodu bacanja igle kako bi izračunao broj π.
- Navedeni primjer smatran je najranijim i najzanimljivijim primjerom primjene Monte Carlo metoda, a poznat je pod nazivom **Buffonov problem igle.**



Vježba – Birthday Paradox

- Metoda se bazira na tome da se igla duljine *2l* nasumično baca na ravninu podijeljenu paralelnim pravcima udaljenim jedan od drugog za *2a*, pri čemu je *l* < *a*, što znači da igla ne može sjeći dva pravca istovremeno.
- Može se dokazati da je vjerojatnost da igla siječe neki od pravaca jednaka $\mathrm{P} = \frac{2l}{\pi a}$.
- Naime, neka je **φ** manji ugao kojeg igla zatvara s pravcem i **x** udaljenost središta igle do najbližeg pravca, kao što je prikazano.



Vježba – Birthday Paradox

- Kut **φ** može imati samo vrijednosti iz intervala $[0, \frac{\pi}{2}]$, a **x** iz [0, a].
- Slijedi da je skup elementarnih događaja jednak $\Omega = [0, \frac{\pi}{2}] \times [0, a]$.
- Iz prethodne slike, vidjeli smo da će igla sjeći pravac samo u slučaju kada je $x < l \sin \varphi$.
- U situaciji kada vrijedi jednakost igla će samo dirati pravac.
- Neka je ***A*** događaj koji predstavlja realizaciju da je igla presjekla pravac, odnosno

$$A = \{Igla\ siječe\ pravac\} = \{(\varphi, x) \in \Omega : x < l\ sin\ \varphi\}.$$

- Primjenom geometrijske vjerovatnosti dobiju se sljedeći rezultati

$$\lambda(A) = \int_0^{\frac{\pi}{2}} l\ sin\varphi\ d\varphi = l$$

$$P = P(A) = \frac{\lambda(A)}{\lambda(\Omega)} = \frac{l}{\frac{a\pi}{2}} = \frac{2l}{a\pi}.$$

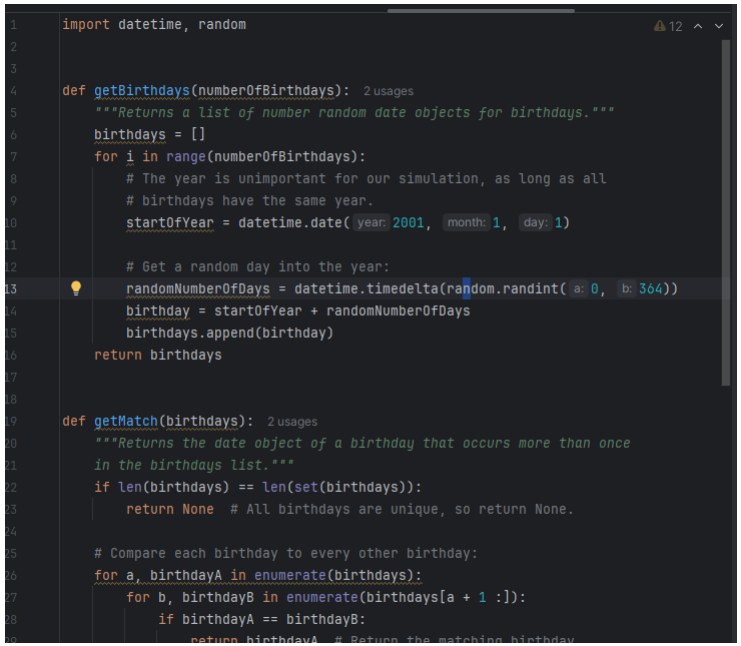
Vježba – Birthday Paradox

- Budući da vjerojatnost može biti procijenjena i kao omjer ukupnog broja bacanja u kojemu je igla pogodila linije i ukupnog broja bacanja, vrijednost broja $\boldsymbol{\pi}$ se tada može računati kao $\pi = \frac{2l}{Pa}$.
- Ljudi su tijekom povijesti stvarno izvodili ovaj pokus.
- Sam Buffon je 1777. bacio iglu **10000** puta i dobio $\boldsymbol{\pi \approx 3.15}$.
- Jedan od najuspješnijih ”bacača” bio je talijanski matematičar Mario
- Lazzarini koji je 1901. bacio iglu ”samo” 3408 puta i dobio tačnost broja π u šest znamenki.
- Njegova igla bila je dugačka 2.5 cm, dok je dužina između pravaca bila 3 cm.
- Broj bacanja u kojima je igla pogodila pravce iznosio je 1808 pa je aproksimacija broja π dobivena iz:

$$\frac{1808}{3408} = \frac{2 \times 2.5}{3 \times \hat{\pi}} \Rightarrow \hat{\pi} = \frac{355}{113} = 3.1415929204.$$

Vježba – Birthday Paradox

- Danas ćemo pokazati koncept koji ćemo zapravo koristiti u sljedećoj laboratorijskoj vježbi.
- Ne brinite ako ne razumijete sve odmah.
- Otvorite document, i ubacite **bdayparadox.py** u svoj PyCharm.
- Pokrenite simulaciju…
- Ukucajte **traženi** broj…



```
import datetime, random


def getBirthdays(numberOfBirthdays):
    """Returns a list of number random date objects for birthdays."""
    birthdays = []
    for i in range(numberOfBirthdays):
        # The year is unimportant for our simulation, as long as all
        # birthdays have the same year.
        startOfYear = datetime.date(year=2001, month=1, day=1)

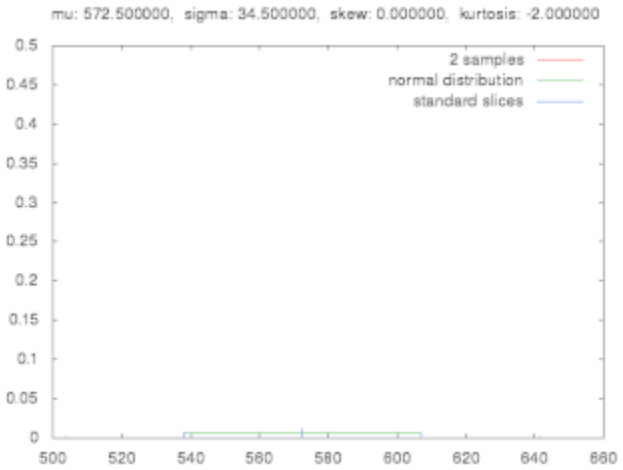
        # Get a random day into the year:
        randomNumberOfDays = datetime.timedelta(random.randint(a=0, b=364))
        birthday = startOfYear + randomNumberOfDays
        birthdays.append(birthday)
    return birthdays


def getMatch(birthdays):
    """Returns the date object of a birthday that occurs more than once
    in the birthdays list."""
    if len(birthdays) == len(set(birthdays)):
        return None  # All birthdays are unique, so return None.

    # Compare each birthday to every other birthday:
    for a, birthdayA in enumerate(birthdays):
        for b, birthdayB in enumerate(birthdays[a + 1 :]):
            if birthdayA == birthdayB:
                return birthdayA  # Return the matching birthday
```

Vježba – Birthday Paradox

- **Šta je Monte Carlo simulacija?**
- Monte Carlo simulacija je računarska metoda koja koristi ponavljano slučajno uzorkovanje za dobivanje numeričkih rezultata, modelirajući neizvjesnost u sistemima.
- Ponavljanjem hiljadu simulacija s različitim ulaznim vrijednostima, ona predviđa vjerojatnost ishoda, a primjenjuje se u financijama (rizik), fizici, inženjerstvu i umjetnoj inteligencij.



Vježba – Birthday Paradox

- **Zašto se koristi slučajnost?**
- Slučajnost se koristi za modeliranje nepredvidivih pojava, postizanje nepristranosti (npr. u izvlačenjima ili istraživanjima) te u simulacijama i kriptografiji, gdje su nepredvidivi ishodi ključni za sigurnost i tačnost.
- Ona omogućuje rješavanje problema gdje uzroci nisu mjerljivi ili kontrolirani, često se temeljeći na zakonima vjerojatnosti

Vježba – Birthday Paradox – IN CLASS

- **ZADATAK: Analiza stvarnih podataka o rođendanima**
- Pregledajte listu s popisom rođendana (real_birthdays.txt).

1. Prebrojite koliko ukupno ima rođendana.
2. Prebrojite koliko postoji JEDINSTVENIH rođendana.
3. Odredite:
   - Postoje li duplikati?
   - Koliko duplikata?

Za prebrojavanju unutar Pytona možete koristiti:

```
set()
len()
```

Vježba – Birthday Paradox – IN CLASS

- Koju je vjerojatnost simulacija predvidjela?
- Je li naša stvarna grupa ispitanika imala iste rođendane?
- Potvrđuje li to ili proturječi simulaciji?
- Zašto bi mogle postojati razlike?
- Zašto je vjerojatnost ≠ rezultat pojedinačnog eksperimenta?

Vježba – Birthday Paradox

- Zašto bi vjerovatnost simulacije i stvarno promatranje u učionici moglo biti različito?

Danas ste naučili

- Čitanje datoteka
- Liste - popise
- Sets - skupove
- Jedinstveno brojanje
- Stvarni vs. simulirani podaci
- Varijabilnost uzorkovanja
- Interpretacija

- **To je UVOD U ANALIZU PODATAKA.**

Next week…

- **The Not-So-Basics, advanced Python features**

